

PROMPT IA 13

SRE + AIOps + Diagnostic Kubernetes

K8sGPT + KubeBuddy + Prometheus + Alertmanager + Loki + Tempo + Hubble + Grafana +
Runbooks-as-Code

Mission : construire la couche SRE/AIOps de la plateforme e-commerce afin de reduire le MTTR, standardiser le diagnostic et produire des recommandations exploitables sans autoriser une IA a modifier directement la production.

Observe -> correlate -> diagnose -> propose -> approve -> GitOps -> verify

1. Role de l'IA de developpement

Tu es Principal SRE / Platform Reliability Engineer. Implemente une couche de diagnostic et d'assistance aux incidents qui s'appuie sur les signaux existants de la plateforme. Reutilise les composants deja deployes et evite de creer une seconde stack d'observabilite.

K8sGPT, KubeBuddy ou tout moteur IA sont des assistants de diagnostic. Aucune remediation autonome en production, aucun kubectl apply automatique et aucun contournement de Git, OPA, Kyverno, CI ou FluxCD.

2. Architecture cible

```
Applications / Kubernetes / Platform
|
+--> Prometheus / Thanos ----+
+--> Loki -----+
+--> Tempo -----+--> Correlation / Incident Context
+--> Hubble -----+ |
+--> Kubernetes Events -----+ v
+--> Alertmanager -----+ K8sGPT / KubeBuddy
|
v
Diagnostic structure
|
+-----+-----+
| |
| |
Runbook Proposal
| |
+-----> PR GitOps
|
CI / OPA / Kyverno
|
FluxCD
|
Verify
```

3. Responsabilites

Composant	Responsabilite
Prometheus/Thanos	Metrics, SLI/SLO, capacity et vue multi-site.
Alertmanager	Routing et lifecycle des incidents operationnels.
Loki	Logs applicatifs et plateforme.
Tempo	Traces distribuees OpenTelemetry.
Hubble	Flux reseau Cilium et diagnostic NetworkPolicy.
K8sGPT	Analyse explicable des objets Kubernetes et enrichissement du diagnostic.
KubeBuddy	Diagnostic Kubernetes complementaire, lecture seule par default.
Grafana	Dashboards et vues incident.
Splunk	SIEM et evenements securite cibles.

4. K8sGPT - implementation

Deployer K8sGPT de facon declarative et restreinte. Selectionner uniquement les analyzers utiles et mesurer le bruit produit avant d'en activer davantage.

```
Kubernetes API (read-only)
|
```

```

K8sGPT
|
analyzers cibles
- Pod / Deployment / StatefulSet
- Service / Ingress-Gateway
- PVC / storage
- HPA / PDB
- NetworkPolicy
- events / failures
|
structured finding
|
severity + evidence + probable cause + suggested checks

```

Le resultat doit toujours separer faits observes, hypothese et recommandation. Une recommandation IA ne doit jamais etre presentee comme une cause certaine sans preuve.

4.1 Backend IA et confidentialite

Abstraire le backend IA. Privilegier une option self-hosted lorsque raisonnable. Si un backend externe est utilise, ajouter anonymisation/redaction des secrets, tokens, credentials, PII, payloads sensibles et noms internes lorsque necessaire. Configurer timeout, quota, circuit breaker et audit.

5. KubeBuddy

Integrer KubeBuddy comme outil de diagnostic complementaire. Lecture seule par default. Ses constats doivent etre transformables en rapport structure et rattaches a un incident, service, namespace et cluster.

```

KubeBuddy
-> inspect
-> findings
-> evidence
-> recommended runbook
-> optional remediation proposal
-> NEVER direct production mutation

```

6. Incident Context Builder

Construire une fonction/service leger qui rassemble automatiquement le contexte d'un incident a partir d'un alert fingerprint, service, namespace ou trace_id.

```

buildIncidentContext(input)
-> Alertmanager alert
-> recent deployments / Flux revision
-> pod health / events
-> Prometheus SLI
-> Loki error window
-> Tempo traces
-> Hubble flows
-> Kafka/RabbitMQ/PostgreSQL signals if relevant
-> K8sGPT/KubeBuddy findings
-> compact incident bundle

```

Factoriser les collectors. Chaque collector doit avoir timeout, limite de volume, erreurs explicites et tests.

7. Correlation

Correlier les incidents avec correlation_id, trace_id, deployment revision, image digest, cluster, site, namespace, service et time window. Ne pas utiliser une correlation temporelle seule comme preuve de causalite.

8. Runbooks-as-Code

Versionner les runbooks dans Git. Chaque alerte actionable doit pointer vers un runbook exploitable.

```

runbooks/
kubernetes/
pod-crashloop.md
pending-pod.md

```

```
pvc-full.md
kafka/
consumer-lag.md
broker-unavailable.md
postgres/
failover.md
replication-lag.md
rabbitmq/
queue-backlog.md
network/
denied-egress.md
certificates/
expiration.md
```

Chaque runbook contient : symptômes, impact, prerequis, diagnostic, commandes read-only, criteres de decision, remediation via GitOps, rollback, verification et escalade.

9. Alerting oriente SLO

Conserver la strategie existante : Alertmanager pour incidents operationnels, Splunk pour securite/SIEM. Ajouter burn-rate alerts et deduplication/grouping afin de reduire le bruit.

```
SLO breach / burn rate
|
Alertmanager
|
incident context
|
diagnosis + runbook
|
human decision
```

10. Dashboards SRE

Fournir les dashboards-as-code suivants : Cluster Overview, Service Health, SLO/Burn Rate, Kafka, PostgreSQL, RabbitMQ, Redis, OpenSearch, Network/Hubble, Certificates, Deployment/Flagger et Incident Investigation.

Les dashboards doivent favoriser les symptomes et dependances critiques. Eviter les panneaux decoratifs ou les centaines de metrics sans decision associee.

11. Diagnostic reseau

Utiliser Hubble/Cilium pour determiner les flux refuses, pertes, DNS et NetworkPolicies. Croiser avec Istio Ambient et Squid pour les problemes egress.

```
Service failure
-> Hubble flow
-> Cilium policy
-> Istio waypoint/egress
-> Squid allowlist/log
-> external dependency health
```

12. Diagnostic Kafka / RabbitMQ / PostgreSQL

Ajouter des playbooks et collectors specialises pour consumer lag, ISR/broker health, MirrorMaker2, queue backlog/DLQ, PostgreSQL replication/failover, PgBouncer et saturation storage. Les diagnostics doivent rester compatibles avec les operators Strimzi, RabbitMQ Cluster Operator et CloudNativePG.

13. Certificats et PKI

Integrer les alertes expiration/renouvellement deja definies. Warning sous 30 jours, critical sous 7 jours et critique immediate sur echec de renouvellement/rotation. Couvrir certificats publics, internes et CA/intermediaires.

14. Remediation proposee, jamais autonome

Finding
-> proposed change
-> human review
-> branch
-> PR
-> CI
-> OPA / Kyverno
-> approvals
-> FluxCD
-> post-change verification

Pour Tier 0, respecter la double approbation. Une suggestion peut generer un patch ou une PR candidate, mais ne doit jamais merger ni deployer seule.

15. Boucle de verification

Toute remediation doit declencher une verification apres deploiement : etat FluxCD, rollout, SLI, logs, traces et disparition du symptome. Si le signal se degrade, appliquer le mecanisme de rollback prevu plutot que poursuivre aveuglement.

16. Securite et RBAC

Creer des ServiceAccounts dedies avec droits read-only minimaux pour K8sGPT/KubeBuddy. Interdire secrets get/list lorsque non indispensable. Les acces aux observability backends sont scopes par namespace/service lorsque possible. Auditer chaque execution et chaque requete IA.

17. Protection des donnees

Avant envoi vers un moteur IA, filtrer secrets, Authorization headers, cookies, tokens, PAN/CVV, credentials PSP, donnees personnelles non necessaires et contenu client. Conserver uniquement le contexte utile au diagnostic.

18. Performance et cout

Eviter les analyses IA permanentes sur tout le cluster. Privilegier une execution event-driven sur alerte ou a la demande, avec cache court, deduplication et limite de concurrence. Mesurer latence, taux de succes et cout/ressources du backend IA.

19. Tests obligatoires

Scenario	Attendu
CrashLoopBackOff	Diagnostic contient events/logs pertinents et runbook, sans mutation.
NetworkPolicy deny	Hubble/Cilium identifie le flux refuse.
Kafka lag	Contexte consumer/broker/partition et runbook correct.
PostgreSQL failover	Etat CNPG et impact applicatif correles.
RabbitMQ backlog	Queue, consumers, DLQ et saturation visibles.
Certificate expiry	Alerte et runbook de rotation corrects.
Bad deployment	Correlation avec revision/image digest et SLI.
Secret in log	Redaction avant transmission au backend IA.
AI unavailable	Diagnostic classique reste disponible.
Suggested fix	Aucun changement prod sans PR/approbation/FluxCD.

20. Chaos validation

Utiliser Chaos Mesh pour injecter des pannes controlees et verifier que les alertes, correlations, diagnostics et runbooks sont corrects. Commencer en environnement non-production puis executer des game days controles.

```
fault injection
-> alert
-> incident context
-> K8sGPT/KubeBuddy diagnosis
-> runbook
-> remediation path
-> recovery verification
-> postmortem
```

21. Postmortems

Generer un squelette de postmortem a partir des faits collectes, sans inventer la root cause. Le document doit separer timeline factuelle, impact, detection, cause confirmee, facteurs contributifs, remediation et actions preventives.

22. SRE metrics

```
MTTD
MTTA
MTTR
alert_noise_ratio
false_positive_rate
runbook_success_rate
diagnostic_duration
autodiagnostic_success_rate
change_failure_rate
rollback_rate
SLO burn rate
```

Comparer avant/apres l'introduction des assistants AIOps afin de prouver qu'ils reduisent effectivement le temps de diagnostic.

23. Integration Backstage / Platform API

Exposer dans Backstage les liens vers dashboards, SLO, runbooks et dernier diagnostic. La Platform API peut declencher une analyse read-only ou produire une proposition de changement, toujours soumise a Authorization-as-Code et au workflow GitOps.

24. CI/CD et GitOps

Versionner Helm/Kustomize, dashboards, alerts, recording rules, runbooks, RBAC, policies, K8sGPT configuration et integrations. Valider schemas, lint, tests et policies en CI avant FluxCD.

25. Ordre d'implementation

1. RBAC read-only + namespaces
2. K8sGPT minimal sur environnement test
3. KubeBuddy read-only
4. Incident Context Builder
5. Runbooks-as-Code
6. Alertmanager links / routing
7. Grafana incident dashboards
8. Hubble/network diagnostics
9. Kafka/PostgreSQL/RabbitMQ collectors
10. Redaction/privacy layer
11. PR proposal workflow
12. Chaos validation
13. Backstage integration
14. Multi-site validation
15. Measure MTTD/MTTR improvements

26. Definition of Done

- Une alerte critique peut être transformée en incident context structure en quelques secondes.
- K8sGPT et KubeBuddy fonctionnent en lecture seule avec RBAC minimal.
- Les secrets et données sensibles sont filtrés avant tout backend IA.
- Chaque alerte actionnable importante possède un runbook versionné.
- Les diagnostics couvrent Kubernetes, réseau, Kafka, PostgreSQL et RabbitMQ.
- Une proposition corrective suit PR -> CI/policies -> approbation -> FluxCD.
- Aucune IA ne peut modifier directement la production.
- Les tests Chaos démontrent la qualité des alertes et diagnostics.
- Backstage expose SLO, dashboards, runbooks et contexte incident.
- Les gains MTTD/MTTR sont mesurés et comparables.

Le succès du Prompt 13 ne se mesure pas au nombre de composants IA déployés. Il se mesure à la réduction du bruit, à la qualité des diagnostics, au MTTR et au maintien d'un contrôle humain/GitOps strict sur toute remédiation.